

Problem Set 2

Hoa T. Vu

PyTorch Basics

Tensors and Autograd

PyTorch tensors with `requires_grad=True` track every operation so that gradients can be computed automatically via `.backward()`.

```
import torch

x = torch.tensor(3.0, requires_grad=True)
y = x**2 + 2*x + 1          # y = (x+1)^2

y.backward()                 # compute dy/dx
print(x.grad)                # tensor(8.)
```

For multi-dimensional parameters, `.backward()` fills `.grad` for every leaf tensor with `requires_grad=True`.

Gradient Descent Loop

The standard pattern for one gradient descent step:

```
# 1. forward pass: compute loss
loss = ((y - y_hat)**2).mean()

# 2. backward pass: compute gradients
opt.zero_grad()    # clear gradients from the previous step
loss.backward()    # accumulate gradients into .grad

# 3. update parameters
opt.step()
```

Fitting a Linear Model with PyTorch

The example below fits $\hat{y} = \theta_0 x + \theta_1$ to seven data points using SGD.

```
import torch

X = torch.tensor([1, 2, 3, 4, 5, 6, 7], dtype=torch.float)
y = torch.tensor([38, 47, 53, 58, 63, 60, 62], dtype=torch.float)

theta = torch.zeros(2, requires_grad=True)
opt = torch.optim.SGD([theta], lr=1e-3)

for _ in range(10_000):
    y_hat = theta[0]*X + theta[1]
```

```

    loss = ((y - y_hat)**2).mean()
    opt.zero_grad()
    loss.backward()
    opt.step()

print(theta)    # learned coefficients
print(loss)     # final MSE

```

Key points:

- `torch.zeros(3, requires_grad=True)`: three parameters initialized to 0.
- `opt.zero_grad()`: must be called before each `.backward()` to reset accumulated gradients.
- `loss.backward()`: computes $\partial\mathcal{L}/\partial\theta_i$ for all i .
- `opt.step()`: applies $\theta \leftarrow \theta - \alpha \nabla_{\theta}\mathcal{L}$.

Problems

Problem 1: Cubic Fitting (PyTorch). Fit the model $\hat{y} = at^3 + bt^2 + ct + d$ to the data below using **PyTorch** and gradient descent.

t	y	t	y
-3.00	-76.39	0.43	-3.77
-2.57	-55.41	0.86	-3.61
-2.14	-34.20	1.29	3.21
-1.71	-21.28	1.71	8.84
-1.29	-18.66	2.14	16.65
-0.86	-12.17	2.57	32.36
-0.43	-6.37	3.00	49.94
0.00	-5.63		

- Initialize a, b, c, d as `torch.tensor` with `requires_grad=True`.
- Define the MSE loss $\mathcal{L} = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$.
- Run gradient descent for at least 1000 iterations with a suitable learning rate.
- Report the final values of a, b, c, d and the final loss.
- Plot the data points and the fitted curve.

Problem 2: Cubic Fitting from Scratch. Repeat the fitting task from Problem 1, but implement gradient descent **without any libraries** — only built-in Python (lists, arithmetic, `math`).

- (a) Derive the partial derivatives $\frac{\partial \mathcal{L}}{\partial a}, \frac{\partial \mathcal{L}}{\partial b}, \frac{\partial \mathcal{L}}{\partial c}, \frac{\partial \mathcal{L}}{\partial d}$ analytically.
- (b) Implement the update rule using only Python lists and loops. Run for at least 1000 iterations.
- (c) Report the final values of a, b, c, d and the final loss.
- (d) Compare your result to the PyTorch solution from Problem 1. Do they converge to the same values?

Problem 3: Subgradients.

(a) **ReLU:** $f(x) = \max(0, x)$

- Write down $\partial f(x)$ for all $x \in \mathbb{R}$.
- What value is conventionally used at $x = 0$ in practice, and why?

(b) **SCReLU:** $f(x) = (\text{clip}(x, 0, 1))^2$, $\text{clip}(x, a, b) = \max(a, \min(b, x))$

- Write down $f(x)$ as a piecewise function.
- Write down $\partial f(x)$ for all x . Identify every point of non-differentiability and give the full subdifferential there.

Problem 4: Cross-Entropy Loss. Binary cross-entropy for true label $y \in \{0, 1\}$ and predicted probability $\hat{y} = \sigma(\theta^\top x)$:

$$\ell(\theta) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

- (a) **Behavior.** What happens to ℓ as $\hat{y} \rightarrow 0$ and $\hat{y} \rightarrow 1$ when $y = 1$? Repeat for $y = 0$. Sketch the loss as a function of $\hat{y}$ for each case.
- (b) **Gradient.** Compute $\nabla_{\theta} \ell(\theta)$. *Hint:* chain rule, and $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Simplify as far as possible.